Install guide

Signal Explorer | Windows source installation
Version 0.1.0 - Multi-channel CAN and live TX lists
Document revision 1.5 - November 2, 2026

This guide installs the current Python desktop application on another PC. A standalone installer will be addressed later. You need internet access for the initial Python package downloads and any required driver downloads.

1. Prepare the PC and project files

Install **64-bit Python 3.12** and its Python launcher. This is the version used on the development PC. The project declares Python 3.11 or later, but other versions have not been validated here.

Copy or clone the project into a writable folder, for example **C:\Tools\Signal Explorer**. Include **pyproject.toml**, **README.md**, and the complete **src** folder. Copy **examples** for the demo DBC and **tests** if you want to run checks. Do not copy the old **.venv**; create one on the new PC. Git is optional if you copy the files.

Keep the complete src/signal_explorer/assets and scripting folders: the package includes the SE window icon, startup splash and built-in demo script. Copy examples/scripts/mcu for the companion scripting tutorials. No additional runtime package is required for File Viewer or the Python runner.

2. Install the application

Open PowerShell in the project root - the folder containing pyproject.toml - and run:

```
cd "C:\Tools\Signal Explorer"
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install .
```

Using the virtual environment executable directly avoids the need to activate it or change PowerShell script-execution settings.

3. Launch Signal Explorer

```
.\.venv\Scripts\python.exe -m signal_explorer
```

Run this command from the project root whenever you want to start the application. SignalExplorer.bat resolves its own folder and launches that folder's .venv interpreter; it does not require activation or a hard-coded PC path.

Packages installed automatically

PySide6-Essentials - desktop UI; **python-can** - CAN interfaces; **pyserial** - serial adapters; **cantools** - DBC decoding; **ldfparser** - LIN descriptions; **uptime** - timestamp support; **asammdf** via python-can[mf4] - ASAM MDF4 recording and playback. pip also installs dependencies such as NumPy and pandas. Build tooling includes setuptools. These are application dependencies, separate from vendor hardware drivers.

Adapter requirements

Install only the software needed for the adapters you plan to use.

Install vendor-supplied drivers/APIs matching this 64-bit installation before connecting hardware, unless the installer requests USB connection. Keep bus wiring disconnected during setup. Driver installers may need administrator approval. Vendor DLLs are not included in the source.

Virtual CAN, LIN and Ethernet - no hardware

No vendor software is required for the simulators. Choose a virtual adapter in the relevant tab; demo traffic/peers let you test monitoring, sending and logging without physical bus traffic.

CANable - SLCAN firmware

Windows must recognize the adapter as a serial COM port. Install its appropriate USB serial driver if Windows does not configure it automatically. No additional Python packages are required. Choose CANable (SLCAN serial), select its actual COM port, and match the CAN bus bitrate. The new PC may assign a port other than COM3.

CANable - candleLight / gs_usb firmware

This is a different USB connection path from SLCAN. Install the additional backend:

```
.\.venv\Scripts\python.exe -m pip install "python-can[gs-usb]"
```

A compatible USB driver and libusb backend may also be needed on Windows. Configure those for the device's firmware, then choose CANable / candleLight (gs_usb). This path has not been physically validated in this project; the tested CANable connection is SLCAN.

Vector - VN16xx / CANXL adapter option

Install Vector hardware drivers **and the 64-bit XL Driver Library**. The USB driver alone is insufficient. Signal Explorer checks PATH and the standard Public Documents/Vector/XL Driver Library*/bin folder. If detection fails, use **Vector DLL** to select the installed **vxlapi64.dll**, then restart. Select the detected serial number and physical CAN channel; no application assignment in Vector Hardware Config is required.

Vector XL Driver Library

Kvaser - including U100-C

Install Kvaser Windows drivers with the CANlib runtime. Select Kvaser (U100 / CANlib), then Refresh devices. Channel indices start at zero. A listed Kvaser virtual channel is not a physical U100-C.

PEAK - including PCAN-USB Pro FD

Install PEAK hardware drivers and the 64-bit **PCAN-Basic API** (PCANBasic.dll). Select the PEAK option and a detected CAN channel. Manual names include PCAN_USBBUS1 and PCAN_USBBUS2; use detected names when several adapters are attached.

For **LIN**, separately install PEAK LIN drivers, **PLIN Manager** and 64-bit **PLIN-API** (PLinApi.dll). PCAN-Basic alone does not supply LIN. Select LIN > PEAK PLIN / PCAN-USB Pro FD, Refresh, then a detected LIN channel. PCAN-USB Pro/Pro FD and PLIN-USB families use this API; a CAN-only PCAN-USB cannot supply LIN. Native PEAK LIN is experimental and awaits a physical bench test.

Intrepid - ValueCAN / neoVI

Install Intrepid hardware drivers and its native 64-bit **icsneo40.dll**, then install the optional Python dependencies:

```
.\.venv\Scripts\python.exe -m pip install ".[intrepid]"
```

This adds python-ics and filelock. Select the Intrepid adapter by serial number, not its COM port. For ValueCAN4-2, HSCAN is CAN1 and HSCAN2 is CAN2.

Verify and troubleshoot

A short check before connecting to a live CAN bus.

1. Verify the software without hardware

Launch Signal Explorer, select **Virtual**, enable **Inject demo traffic**, and click **Connect**. The trace should show ID 100 with changing data. Load **examples/demo.dbc** to display the demo message name and decoded signals.

CAN Auto-scroll defaults unchecked: latest data per channel and ID. Check it for timestamped history. History retains the most recent 10,000 frames. **Pause Exploring** freezes trace updates while staying connected; **Start Exploring** resumes. Frames received while paused are not replayed.

2. Verify hardware reception

Connect the adapter by USB, choose its backend and channel, set the correct bus bitrate, and click Connect. Use a powered, correctly wired and terminated CAN bus with a device that is transmitting. Check the RX count. Click Clear filter if frames might be hidden. Merely connecting USB does not generate CAN traffic.

Common issues

Python or py is not recognized: verify Python 3.12 is installed and reopen PowerShell. Check with **py -3.12 --version**.

No module named signal_explorer: run the install command from the project root and launch using the same .venv Python executable.

COM port access denied: disconnect that port in other Signal Explorer windows and close other programs using it. Only one process can hold the CANable serial port.

Connected, RX stays at zero: confirm the bus has traffic, the bitrate matches, CAN-H/CAN-L wiring is correct, and the required termination is present. A quiet bus can connect successfully without producing trace rows.

RX increases but the trace appears empty: click Clear filter and ensure exploring is running. While paused, trace counters and rows stay frozen.

Missing DLL or no detected channels: install the vendor's 64-bit API as well as its USB driver. Reconnect USB, use Refresh devices, or restart after changing DLL settings. No attached device can legitimately produce an empty channel list.

Optional automated checks

For optional checks, copy tests and install the test dependencies with pip install ".[dev,intrepid]" in this .venv. From the project root run:

```
.\.venv\Scripts\python.exe -m pytest tests -q
```

196 automated checks pass, including adapter preferences, DBC editing and NI-XNET mocks. USB-8502 traffic, Ethernet TX and powered ValueCAN LIN/Ethernet await bench tests.

Logging and playback

Click **Start Logging** to record RX and TX; **Stop Logging** finalizes the MDF4 file and opens a save dialog. Cancelling retains it for **Save Log...**. Logging continues while the trace is paused or filtered. **Playback Log as TX** replays a selected recorded bus/direction at best-effort 1x timing through the connected adapter; **Stop Playback** stops it. Physical timing and third-party interoperability are not yet qualified.

Current scope

CAN supports Classic/FD. LIN offers simulated Monitor/Master/Slave and experimental native Intrepid, Vector XL and PEAK PLIN. Ethernet now offers an isolated simulator, Windows PC Ethernet through Npcap, and ValueCAN4-2EL Ethernet. Sending, repeat, filters and MF4/replay are available. Ethernet setup is on pages 5-6. SENT, CAN XL and a packaged installer remain future work.

DBC files are optional; copy the files you intend to use separately. No compiler is normally needed when compatible prebuilt Python wheels are available. Record working package versions for reproducible deployments; the current project uses minimum version constraints rather than a pinned dependency lock.

LIN adapter setup

Monitor, Master and Slave | native drivers are experimental until bench validation.

Intrepid ValueCAN4-2EL

Install the Intrepid drivers, 64-bit **icsneo40.dll**, and the optional Python dependencies shown on page 2. In the LIN tab select **Intrepid ValueCAN4-2EL**, Refresh, then its serial number and LIN 1. The app checks the actual device type: ValueCAN4-2 and ValueCAN3 are CAN-only and will not appear here. The development unit VE0245 was detected and its settings read over USB; physical LIN traffic has not been tested.

Baud rate and the master resistor are configured for the selected role through temporary settings, with no EEPROM writes. Disconnect restores the saved settings and closes the native handle, leaving LIN communication off until a new session enables it. **Wake is disabled for Intrepid** because its legacy Python binding has no verified dedicated wake-pulse operation; use the external master to wake a sleeping bus.

Vector VN1640 with LIN piggies

Use the installed Vector hardware drivers and XL Driver Library from page 2. In LIN select **Vector VN16xx / XL LIN** and Refresh. Select the serial, physical LIN channel and piggy shown by the driver. The VN1640 requires LIN piggies installed and configured. A VN1610 with CAN transceivers is not a LIN interface. Release initialization access in CANalyzer or other XL applications before connecting this channel.

PEAK PLIN

Install PEAK LIN drivers, PLIN Manager and the matching 64-bit PLIN-API as described on page 2. Select **PEAK PLIN / PCAN-USB Pro FD** and Refresh. PCAN-Basic is the separate CAN API; it does not provide LIN. Close other PLIN clients on the selected LIN channel.

Choose a role and transmit

Monitor: no local responses or header requests. **Master:** Send Header requests one defined ID; Start Schedule cycles the configured slots. Selected local response data can be transmitted with the header. **Slave:** select local publishers and Apply Responses; the adapter waits for the external master header before responding. Response updates are buffered in hardware, not generated from a GUI callback.

Load the matching LDF, baud rate, frame lengths and checksum model on both nodes. Pause Exploring freezes only the trace; stop the schedule or playback separately. Start Logging records RX/TX independently of trace filters. Master playback replays headers with current response configuration; Slave playback updates the selected response buffers at recorded times.

Device ownership and powered bench check

Signal Explorer prevents concurrent CAN/LIN on the same Vector and CAN/LIN/Ethernet on the **same Intrepid device**. Disconnect that device's existing session first. Different devices may remain connected in separate tabs; shared native multi-protocol sessions are future work.

USB alone is sufficient for detection, not a powered LIN bus test. Follow the vendor connector pinout, supply requirements and common-ground wiring, with exactly one master and the proper LIN master pull-up. LIN does not use CAN termination. Verify responder removal, RX/TX direction, checksum/error reporting, timing, sleep/wake and MF4 import on the real bench. Mock tests cannot qualify these physical behaviors.

Ethernet adapter setup

PC Ethernet through Npcap | Intrepid ValueCAN4-2EL | simulation stays available

1. Install PC Ethernet capture support

Install current **Npcap** from its official installer. This is a separate Windows driver/runtime, not a Python package and not bundled with Signal Explorer. The app loads **%SystemRoot%\System32\Npcap\wpcap.dll**; it does not use a DLL copied into the project. No Scapy or additional Python capture package is required.

Npcap download and installation instructions

Follow your Npcap capture-permissions policy. If installed with administrator-only access, opening an adapter may require an elevated app session. Normal capture permissions otherwise suffice. Raw Wi-Fi/monitor-mode support is not needed for this release: Wi-Fi and loopback are not offered. The app does not change PC IP addresses, link speeds or duplex.

2. Install ValueCAN4-2EL support

Use the same Intrepid Windows driver, matching 64-bit **icsneo40.dll**, and optional packages as CAN/LIN. On the new PC run:

```
.\.venv\Scripts\python.exe -m pip install ".[intrepid]"
```

Use current python-ics with the vendor's Ethernet buffer fixes (1!26.3.9+ recommended; this build used 1!26.3.17). Copying the project does not copy installed native drivers. Select the actual ValueCAN4-2EL serial; CAN-only ValueCAN4-2 and ValueCAN3 are not Ethernet adapters.

Provide the vendor-specified **external 12 V** and matching Ethernet breakout cable. USB detection alone is insufficient for Ethernet. The 2EL has a 10/100 PHY; preserve/configure its speed/duplex in neoVI Explorer, then release that tool before app connection. Signal Explorer enables Ethernet temporarily in RAM, restores settings on close and never writes EEPROM.

ValueCAN4-2EL power and Ethernet specifications

3. Discover and select

Open **Ethernet**: first display scans adapters. **Refresh adapters** rescans while disconnected. The list always keeps **Simulated Ethernet**, plus detected PC Ethernet interfaces and 2EL units. PC entries show name, MAC and up/down state; virtual Ethernet interfaces are included. Interfaces can remain listed but disabled when Npcap/capture access is unavailable.

Select the intended port and Connect Ethernet to start capture. Discovery itself never opens a session or transmits. Optional **Promiscuous capture** applies only to PC ports; a switch mirror/TAP is needed to observe other switch ports. Disconnect CAN/LIN sessions on the same ValueCAN before opening its Ethernet session.

4. Check setup without transmission

Use a known transmitting peer to confirm passive RX and link state. Clear filters and Start Exploring if needed. Simulation needs no driver: select it, enable demo traffic/peer, connect and inspect test frames. Demo traffic and synthetic ACK replies stay inside the simulator; they never run on hardware.

Ethernet verification & recovery

Confirm the selected interface and expected traffic before sending or replaying.

Simulation acceptance check

Choose Simulated Ethernet and Connect Ethernet with demo traffic and peer enabled. Expect UDP telemetry, ARP, VLAN UDP and raw 88B5 test frames. Select VLAN UDP, Load Test Frame and Send once: expect local TX and a synthetic ACK RX. Enter 100 in Interval (ms) to repeat; Send becomes Stop. Inject Test RX is simulation-only.

Hardware capture and controlled transmit check

Choose the exact PC port or externally powered 2EL and Connect Ethernet. A session-open status is not proof of physical link. Verify peer traffic first. Load/edit a suitable test frame, including destination MAC and payload for the bench, then Send once and confirm bytes at the independent peer before repeat/playback. Physical TX acceptance remains pending for this build.

PC test presets use the selected NIC source MAC; their IP/destination values remain example addresses. ValueCAN raw frames use the editor source MAC. No automatic ARP resolution, TCP session, DoIP activation line, DoIP/XCP stack or PC IP configuration is provided. Hardware Send/repeat/playback cause real transmission on the selected port.

Logging and playback

Start Logging records RX/TX before display filters/pause; Stop Logging prompts for an .mf4 name. Cancel retains it for Save Log.... Playback Log as TX, or Playback Log as Simulated TX on the virtual adapter, selects recorded bus/direction and replays at best-effort 1x timing. Playback stops repeated sending. Old Ethernet simulation logs still work; unrelated CAN/LIN logs do not.

Ethernet stores raw frames in an MDF 4.10 container using custom **SignalExplorer_EthernetFrame** records. It is not standardized ASAM Ethernet bus-event interchange or independent conformance certification. TX means driver/API acceptance, not peer receipt. FCS is excluded, while NIC-provided padding may be absent and is added when replay requires minimum TX length.

Common Ethernet issues

Capture unavailable: install/check Npcap and permissions; Refresh adapters. **No ports:** check Windows device enumeration; Wi-Fi/loopback are intentionally excluded. **ValueCAN in use:** release other CAN/LIN/Vehicle Spy/neoVI sessions on that serial.

No RX: verify cable/link and peer traffic; 2EL also needs 12 V. An ordinary switched port does not see every other port. **Checksum warning:** raw packet edits or NIC checksum offload can be responsible; verify at a peer before blaming the bus.

Driver drops: Npcap capture/interface loss; Intrepid shows n/a. **Display drops:** UI queue overflow, separately from recorder overflow. **Unsupported/error frames:** rejected captures, not logged; includes jumbo/truncated/stacked-VLAN/802.3 length frames or native CRC/length faults. Standard MTU 1500 and one VLAN tag are supported.

No sustained line-rate or lossless capture guarantee is claimed. Native RX/API failures close the session and stop/finalize active operations. Use the full users guide for column definitions, controls, recovery and cable-specific CAN/LIN/Ethernet pinouts.

October 29 update & File Viewer

Revision 1.2 | Python scripting, log viewing and timestamp correction

1. Update an existing source installation

Close Signal Explorer and its editor/viewer windows, preserving any unsaved scripts and logs. Replace/update the project files. From the project root, reinstall into the same environment:

```
.\.venv\Scripts\python.exe -m pip install .
# Intrepid users instead use:
.\.venv\Scripts\python.exe -m pip install ".[intrepid]"
```

For an editable development installation use -e . or -e ".[intrepid]". Launch with the same .venv interpreter. Do not run a new GUI alongside an older process holding the adapter. The Python worker uses the application's interpreter; there is no separate scripting executable to install.

2. Confirm startup branding and the menu

The black SE icon has a red S and white E. A separate splash adds white SIGNAL above the mark and EXPLORER below it. It appears during imports/window construction and closes when the main application opens. File Viewer replaces the former Scripts menu; the title-bar icon remains the SE mark.

3. Verify scripting without hardware

Connect Virtual CAN with Inject demo traffic checked. Choose File Viewer > Open CAN Simulation Example and Run. The script sends ID 321 hex with eight bytes; the virtual peer replies on 221 hex with the same bytes. Each verified exchange increments every byte, wraps FF to 00 and repeats until Stop. Pause/Resume/Stop and Script Output are in the separate editor. CAN FD mode also supports the eight-byte echo.

Use File Viewer > Open Python Script... for examples/scripts/mcu/*.py. Five examples use the built-in simulator. The other four require matching MCU firmware and default to a configuration guard before TX. Read Scripting User's Guide.pdf before adapting them. The se context is injected by the app; it is not a pip package.

4. Verify log viewing and replay

Use File Viewer > Open Logfile for a single-protocol CAN/CAN FD, LIN or Signal Explorer Ethernet MF4. Logs open read-only, even offline. Connect the corresponding destination, select Recorded bus and RX only/TX only/RX and TX, then Replay. Pause/Resume/Stop, highlighted rows, Follow execution and Replay Output remain in the viewer. Log line delay changes display refresh only, preserving 1x transmission timing; Python line delay slows actual script execution.

5. Make fresh Intrepid recordings after this update

The app now anchors Intrepid CAN/LIN/Ethernet RX hardware intervals to host time per connection, aligning them with local TX. Earlier recordings could mix a 2007 device-clock epoch with the PC clock and flatten subsequent timestamps. Their original timing cannot be recovered. Restart, reconnect and record a new file. Replay detects selected gaps over 24 hours before any TX and bounds platform waits to prevent timeout overflow. No Skip gaps control is provided.

Runtime versus documentation tooling

The tutorials use only Python's standard library and the existing app CAN API. No CAPL compiler, external editor, UDS library or CAN transport package is added. Full UDS/ISO-TP, LIN/Ethernet scripting and a standalone installer remain future work. reportlab and pypdf are needed only to regenerate the manuals, not to run Signal Explorer.

NI-XNET / USB-8502 setup

October 30, 2026 | One-port and two-port high-speed CAN / CAN FD

1. Install the NI software before USB connection

Use Windows x64 and the application's 64-bit Python environment. Obtain NI-XNET for your Windows version and USB-8502 from NI. Include NI Measurement & Automation Explorer (MAX), follow the installer's connection prompts and restart if requested. The native driver is separate from the Python binding. NI-CAN, NI-DAQmx and LabVIEW are not needed for this backend.

NI-XNET driver download

2. Add the optional Python binding

Close Signal Explorer. From the project root, install the NI extra using the interpreter that launches the app:

```
.\.venv\Scripts\python.exe -m pip install ".[nixnet]"
# To include both optional vendor bindings:
.\.venv\Scripts\python.exe -m pip install ".[intrepid,nixnet]"
```

Use -e before the quoted package selector for editable development. Installing the binding does not install the NI-XNET Windows driver. Other adapters remain available without this optional extra.

3. Verify the adapter and select the correct port

Connect USB; the USB-8502 does not need external power for CAN. Locate it in NI MAX and note each CAN interface name (for example CAN1 or CAN2) and physical port. Use MAX Bus Monitor with a known-good peer and matching rates to verify RX; close that monitor and other sessions before opening this port in Signal Explorer.

In the CAN tab choose **NI (USB-8502 / NI-XNET)**, then **Refresh devices**. Select the detected model/serial/interface/port, or enter the exact NI MAX CAN name manually. Start with Classic CAN and a matching bitrate. **NI internal termination (120 ohm)** defaults off; enable only if this port needs an end resistor. It cannot be changed while connected.

4. Confirm Classic CAN, then CAN FD

Connect and check a known RX ID. Send eight known bytes and confirm them independently at the peer; local TX only confirms API acceptance. For FD, disconnect, enable CAN FD and match arbitration/data rates, reconnect, then select FD frame and BRS. Start with 500 kbit/s arbitration and 2 Mbit/s data if the peer supports them. USB-8502 revisions with TJA1043T are certified for FD data up to 5 Mbit/s; 8 Mbit/s is experimental. Verify your hardware revision.

The normal CAN trace, filters, DBC, repeat send, Python scripts, MF4 RX/TX recording and timed replay use this connection. First confirm bytes and timing at the peer. This release has mock coverage for NI enumeration, native frame conversion and MF4 recording; physical USB-8502 qualification remains pending.

Troubleshooting and scope

Python API unavailable: install .[nixnet] in this .venv. **Scan/DLL error:** install or repair 64-bit NI-XNET. **No channels:** check USB and NI MAX; Refresh. **Connect/rate failure:** check the exact CAN name, peer rates and competing sessions. **Bus-off:** stop TX, repair wiring/termination/peer ACK, then disconnect/reconnect.

USB-8473 / NI-CAN is excluded. Only high-speed/FD NI interfaces are enumerated; other NI-XNET models require separate qualification. No NI LIN, Ethernet or synchronization controls are added. See the updated User's Guide for the NI electrical profile, pinout revision differences and detailed bench checks.

NI Python requirements and installation

DBC editor & adapter defaults

October 31, 2026 | Source update and offline verification

1. Update the existing installation

Close Signal Explorer, preserving unsaved scripts, DBC changes and logs. Update the source and reinstall from the project root with the same .venv interpreter. The DBC editor uses the existing Qt and cantools packages; no new runtime dependency, external editor, license server or vendor driver is required for offline editing.

```
.\.venv\Scripts\python.exe -m pip install .
# Preserve both optional vendor integrations when needed:
.\.venv\Scripts\python.exe -m pip install ".[intrepid,nixnet]"
```

Use -e before the package selector for an editable development install. Keep the complete src/signal_explorer/dbc and ui folders when copying source. The companion scripting guide now also covers enabled-channel selection in test code.

2. Adapter selection on launch

Adapter dropdowns are alphabetical in CAN, LIN and Ethernet. On the first run of each protocol, the virtual adapter is selected even if hardware is connected. Later launches remember that protocol's last selected adapter for this Windows user profile. CAN/LIN remember the backend family; Ethernet remembers the NIC/device identity, not its list index.

Restoring a selection never connects automatically. Use Refresh to discover current hardware and select the channel and bus parameters before Connect. If the saved Ethernet device is missing or unavailable, the app selects Virtual while retaining the saved identity for a later Refresh. These preferences are local Qt settings, separate from the project and .venv; copying the project to another PC does not copy them.

3. Confirm the DBC editor without hardware

On the CAN screen load examples/demo.dbc, then click Edit DBC. A separate window should show messages, signals, a payload bit map, generated DBC preview and Test decoding. Double-click a message/signal to view its properties; Validate checks the database. Use New to create a blank database, or Open to edit another file.

For a self-contained test, add an eight-byte standard Classic CAN message, ID 321 hex. Add EngineRPM at start bit 0, length 16, Intel unsigned, factor 0.25, offset 0 and unit rpm. In Test decoding enter 40 1F 00 00 00 00 00 00; expect 2000 rpm. This offline test sends no CAN frame.

4. Check save and decoder behavior

Save As to a temporary .dbc file and reopen it to confirm its definitions. Unsaved edits remain private. Save to the exact file currently loaded on the CAN screen reloads its decoder; Save As to another path does not select that new file for live decoding. Use Load DBC to activate it explicitly. Closing/New/Open prompt to save dirty work; Cancel keeps the editor open.

Limits and recovery

Invalid or overlapping/out-of-payload signals are rejected. Use Undo/Redo for accepted edits. Test decoding needs exactly the selected message's byte length. Motorola uses DBC sawtooth start-bit numbering; do not assume Intel positions. Multiplexing/float definitions are preserved but not configured by the basic forms. Text must fit Windows-1252; serialization can change formatting. The editor does not encode/transmit signal values.

The User's Guide contains the complete editor control map and practical workflow. docs/dbc-editor.md describes its save isolation and module boundaries. Physical adapter qualification is unchanged.

Multi-channel CAN & transmit lists

November 2, 2026 | Update and verify without changing drivers

1. Update the source installation

Close Signal Explorer and save pending files. Update the complete source tree and reinstall with the same .venv interpreter and existing optional extras. No new runtime dependency or vendor driver is required for these features. Existing CANable, Vector, Intrepid, Kvaser, PEAK and NI backends remain available.

```
.\.venv\Scripts\python.exe -m pip install ".[intrepid,nixnet]"
```

Use -e for an editable installation; omit extras you do not use. Keep the new bus/multichannel.py, bus/intrepid_channels.py, bus/transmit.py, ui/can_channels.py and ui/transmit_window.py modules when copying source. The PDF builders require reportlab/pypdf only for documentation generation.

2. Verify settings and channel discovery

On first launch select Virtual; subsequent launches restore the last backend. CAN now remembers the last physical device and enabled subset per backend/device. Choose Enable channels..., check the intended ports, and confirm. All checked channels share bitrate and Classic/FD settings. Unsupported known FD channels are rejected. Connect and choose one TX channel; no broadcast occurs.

Settings are local to the Windows user, separate from .venv and portable TX lists. Missing channels retain their saved preference; if none are available, choose explicitly before Connect. Restoration never connects or sends. The ordinary Channel dropdown replaces the saved subset with one channel. CAN Auto-scroll now defaults unchecked (latest-value rows); LIN/Ethernet defaults are unchanged.

3. Verify the new sender in simulation

Connect Virtual with demo traffic and load examples/demo.dbc. Open Multi-message TX.... New Raw, ID 321, eight bytes 00 01 02 03 04 05 06 07, then now should produce an RX 221 echo. Add EngineData from DBC, set Cyclic and 100 ms, then Start. Edit RPM or bytes while sending and check subsequent frames. Stop All ends scheduled/queued sends.

Save TX List creates an application-specific JSON configuration, including embedded DBC definitions and waveforms. Open TX List restores it without starting traffic. Both buttons also exist under File. Clone remains; cut/copy/paste were removed from this window. MF4 remains the recording/replay format, not the TX-list format.

4. Hardware acceptance and recording scope

VN1630A serial 604069: one standard ID 5A1 frame, A5 5A 01 02 03 04 05 06, was sent CAN1 to CAN2 at 500 kbit/s Classic CAN with both ports enabled; matching RX and no errors were observed. The session was released. Physical FD and new multi-message workflows remain pending. FIRE 3 inventory/routing is mock tested: 16 CAN plus 8 LIN listed; this does not add FIRE 3 LIN transport.

196 automated tests pass. MF4 now preserves enabled CAN source channels and an embedded map. Replay selects one recorded bus and one main TX channel, stopping periodic GUI senders first. Python send/recv can target an enabled label/native channel; see the updated scripting guide. No automatic multi-bus replay or hardware-clock synchronization is added.